

Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра ПЗ

Практична робота №1
з дисципліни «Розробка проєктів засобами платформи .NET»

Виконав: ст. 1ПІ-24Б

Перевірів: викл.

Кузя Н.І.

Позур М.Ю.

Тема: Базові конструкції C#. Консольний застосунок.

Мета: Закріпити навички роботи з базовими типами даних, операторами, керуючими конструкціями (розгалуження, цикли), масивами та методами мови C#; навчитися організовувати консольний додаток з введенням/виведенням даних та структурувати найпростіший застосунок у вигляді меню з пунктами.

Завдання (Варіант 3. Конвертер величин)

Створити консольний проєкт на базі .NET. Реалізувати меню з повторенням дій до вибору «Вихід». Реалізувати шість напрямків конвертації (три пари одиниць, кожна в обидва боки): кілометри/милі, кілограми/фунти, градуси Цельсія/Фаренгейта. Після вибору напрямку програма запитує значення з валідацією, перевіряє фізичні межі величин та виводить відформатований результат.

Хід роботи

Програму реалізовано з використанням архітектури циклічного текстового меню на основі конструкції `switch expression`. Введення даних захищено за допомогою методів `int.TryParse` та `double.TryParse`, що забезпечує стабільну роботу при некоректному вводі (користувач може вводити як крапку, так і кому). Логіку конвертації винесено в окремі методи, а фізичні обмеження (заборона від'ємної маси або температури нижче абсолютного нуля) реалізовано на етапі обробки вводу.

Лістинг програми (Program.cs)

```
using System;
using System.Globalization;

namespace UnitConverterApp
{
    internal class Program
    {
        private const double KmPerMile = 1.609344;
        private const double KgPerPound = 0.45359237;
```

```

private const double AbsoluteZeroCelsius = -273.15;
private const double AbsoluteZeroFahrenheit = -459.67;

static void Main()
{
    Console.OutputEncoding = System.Text.Encoding.UTF8;

    bool isRunning = true;

    while (isRunning)
    {
        PrintMenu();
        int choice = ReadInt("Ваш вибір: ", 0, 6);

        isRunning = choice switch
        {
            1 => HandleConversion("км", "милі", KilometersToMiles, minValidValue: 0),
            2 => HandleConversion("милі", "км", MilesToKilometers, minValidValue: 0),
            3 => HandleConversion("кг", "фунти", KilogramsToPounds, minValidValue: 0),
            4 => HandleConversion("фунти", "кг", PoundsToKilograms, minValidValue: 0),
            5  =>  HandleConversion("°C", "°F", CelsiusToFahrenheit, minValidValue:
AbsoluteZeroCelsius),
            6  =>  HandleConversion("°F", "°C", FahrenheitToCelsius, minValidValue:
AbsoluteZeroFahrenheit),
            0 => false,
            _ => true
        };
    }

    Console.WriteLine("\nПрограму завершено. Дякуємо за використання!");
}

static void PrintMenu()
{
    Console.WriteLine("\n=====          КОНВЕРТЕР          ВЕЛИЧИН
=====");
    Console.WriteLine("1. км → мілі");

```

```

        Console.WriteLine("2. милі → км");
        Console.WriteLine("3. кг → фунти");
        Console.WriteLine("4. фунти → кг");
        Console.WriteLine("5. °C → °F");
        Console.WriteLine("6. °F → °C");
        Console.WriteLine("0. Вихід");

        Console.WriteLine("=====");
    }

    static bool HandleConversion(
        string fromUnit,
        string toUnit,
        Func<double, double> convertFunc,
        double minValidValue)
    {
        Console.WriteLine($"n--- Конвертація: {fromUnit} → {toUnit} ---");
        double input = ReadDouble($"Введіть значення ({fromUnit}): ");

        if (input < minValidValue)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[Помилка] Фізично неможливе значення! Для {fromUnit} значення не може бути меншим за {minValidValue:F2}.");
            Console.ResetColor();
            return true;
        }

        double result = convertFunc(input);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"Результат: {input:F2} {fromUnit} = {result:F2} {toUnit}");
        Console.ResetColor();

        return true;
    }

```

```
static double KilometersToMiles(double km) => km / KmPerMile;  
static double MilesToKilometers(double miles) => miles * KmPerMile;
```

```
static double KilogramsToPounds(double kg) => kg / KgPerPound;  
static double PoundsToKilograms(double pounds) => pounds * KgPerPound;
```

```
static double CelsiusToFahrenheit(double celsius) => (celsius * 9.0 / 5.0) + 32.0;  
static double FahrenheitToCelsius(double fahrenheit) => (fahrenheit - 32.0) * 5.0 / 9.0;
```

```
static int ReadInt(string prompt, int min = int.MinValue, int max = int.MaxValue)  
{  
    while (true)  
    {  
        Console.Write(prompt);  
        string? input = Console.ReadLine();  
  
        if (int.TryParse(input, out int value) && value >= min && value <= max)  
        {  
            return value;  
        }  
  
        Console.WriteLine($"Некоректне значення. Введіть ціле число від {min} до {max}.");  
    }  
}
```

```
static double ReadDouble(string prompt)  
{  
    while (true)  
    {  
        Console.Write(prompt);  
        string? input = Console.ReadLine();  
  
        if (input != null)  
        {  
            input = input.Trim().Replace(',', '.');  
  
            if (double.TryParse(input, NumberStyles.Float, CultureInfo.InvariantCulture, out double
```

```

value))
    {
        return value;
    }
}

Console.WriteLine("Некоректний ввід. Будь ласка, введіть дійсне число.");
}
}
}
}
}

```

Посилання на GitHub

Репозиторій із вихідним кодом: <https://github.com/shwballl/vntu-dotnet>

```

===== КОНВЕРТЕР ВЕЛИЧИН =====
1. км → милі
2. милі → км
3. кг → фунти
4. фунти → кг
5. °C → °F
6. °F → °C
0. Вихід
=====
Ваш вибір: 1

--- Конвертація: км → милі ---
Введіть значення (км): 20
Результат: 20,00 км = 12,43 милі

===== КОНВЕРТЕР ВЕЛИЧИН =====
1. км → милі
2. милі → км
3. кг → фунти
4. фунти → кг
5. °C → °F
6. °F → °C
0. Вихід
=====
Ваш вибір: 3

--- Конвертація: кг → фунти ---
Введіть значення (кг): 50
Результат: 50,00 кг = 110,23 фунти

===== КОНВЕРТЕР ВЕЛИЧИН =====
1. км → милі
2. милі → км
3. кг → фунти
4. фунти → кг
5. °C → °F
6. °F → °C
0. Вихід
=====
Ваш вибір: 5

--- Конвертація: °C → °F ---
Введіть значення (°C): -300
[Помилка] Фізично неможливе значення! Для °C значення не може бути меншим за -273,15.

```

Рисунок 1 – Результат роботи програми

Відповіді на контрольні питання

1. Чим відрізняються value type та reference type в C#? Наведіть приклади.

Значеннєві типи (int, double, bool, struct) зберігають дані безпосередньо у виділеній пам'яті (стек), при передачі в метод створюється їх копія. Посилальні типи (string, object, масиви, класи) зберігають у змінній лише адресу на об'єкт, розташований у купі (heap).

2. У чому різниця між switch statement та switch expression?

Оператор switch керує потоком виконання команд за блоками case із обов'язковим break/return. switch expression одразу повертає обчислене значення через лямбда-синтаксис (=>), що робить код більш лаконічним.

3. Як коректно обробити некоректне введення користувача без аварійного завершення програми?

Використанням методів TryParse (наприклад, double.TryParse) у циклі while (true). TryParse повертає логічне значення успіху замість викидання винятків.

4. Які переваги виокремлення логіки в окремі методи порівняно з кодом в Main?

Дотримання принципу єдиної відповідальності, усунення дублювання коду, спрощення тестування та підвищення загальної читабельності програми.

5. Як передаються масиви у методи – за значенням чи за посиланням?

Масив – це посилальний тип. За замовчуванням у метод передається копія посилання. Це дозволяє змінювати елементи масиву всередині методу, але не дає змоги замінити сам масив (без використання модифікатора ref).

6. Чим List<T> відрізняється від масиву і коли варто обрати кожен з них?

Масив має фіксовану довжину. List<T> – динамічна колекція, що автоматично змінює розмір при додаванні елементів. Якщо розмір набору невідомий заздалегідь – обирають List<T>, для фіксованих структур – масив.

7. У чому різниця між ref та out параметрами? Наведіть приклад, де підійде лише один з них.

Обидва передають аргумент за посиланням. Проте змінна для ref має бути обов'язково проініціалізована до виклику, а для out – ні, але метод зобов'язаний присвоїти значення out-параметру. Приклад для out: метод TryParse.

8. Чому рядки (string) в C# вважаються незмінними (immutable)?

Після створення екземпляра `string` його вміст у пам'яті змінити неможливо. Методи типу `Trim/Split` не модифікують оригінал, а повертають новий об'єкт. Це гарантує потокобезпечність.

9. Коли варто використовувати `var`, а коли – явно вказувати тип змінної?

`var` використовують, коли тип змінної очевидний із правої частини (`var list = new List<string>()`). Явний тип краще вказувати, коли тип неочевидний з виклику або потрібна конкретна точність.

10. Чим цикл `for` відрізняється від `while` і `foreach`?

`for` – найкращий для ітерацій за індексом. `while` – коли кількість ітерацій невідома і залежить від умови. `foreach` – для безпечного перебору всіх елементів колекції від початку до кінця.

11. Що станеться, якщо не перевірити порожній масив перед обчисленням середнього значення?

Виникне ділення на нуль (виняток `DivideByZeroException` для `int`, або `NaN` для `double`). Цього уникають перевіркою `if (array.Length == 0)` на початку.

12. Чим `int.Parse` відрізняється від `int.TryParse`?

`int.Parse` викидає виняток, якщо рядок не конвертується в число. `int.TryParse` повертає `false` та записує результат в `out`-параметр, не порушуючи роботу програми.

Висновок: під час виконання практичної роботи №1 було створено консольний застосунок конвертера величин на платформі `.NET` з використанням базових конструкцій мови `C#`. Реалізовано архітектуру циклічного текстового меню на основі `switch expression`, безпечне введення та валідацію числових даних за допомогою `double.TryParse`. Алгоритми конвертацій винесено в окремі методи із застосуванням іменованих констант та реалізовано захист від введення фізично неможливих величин (від'ємних мас/відстаней та температур, нижчих за абсолютний нуль).